

Informe de Laboratorio - Semana 9:

Recursividad

Desafío 2: Generador recursivo de combinaciones (Backtracking)

Grupo: Rodriguez Tomás, Argañin Milagros

1. Introducción y Análisis

El presente informe detalla la resolución del Desafío 2 de la Semana 9, centrado en el diseño e implementación de un algoritmo recursivo puro para generar combinaciones de k elementos tomados de un conjunto de n elementos. Este problema se abordó utilizando el patrón de **backtracking**, que trata de explorar un árbol de decisiones y retrocediendo cuando las ramas no conducen a una solución válida.

2. Diseño Algorítmico y Decisiones de Implementación (Punto b)

La implementación se basó en una recursión pura, estructurada estrictamente bajo el patrón de caso base y caso recursivo para garantizar la claridad y evitar errores de recursión infinita.

- **Caso Base:** Se definió para $k = 0$. En este estado, el algoritmo ha seleccionado exitosamente la cantidad de elementos requerida, por lo que devuelve una lista con una tupla vacía $[]$, representando una única combinación válida.
- **Caso Recursivo:** Para cada posición posible del primer elemento, la función realiza una llamada recursiva sobre el subconjunto de elementos situados a su derecha, solicitando combinaciones de tamaño $k - 1$.

Esta estructura asegura que el orden de los elementos se respete y evita la generación de permutaciones redundantes, cumpliendo con la definición matemática de combinación.

3. Evaluación y Test de Referencia (Punto c)

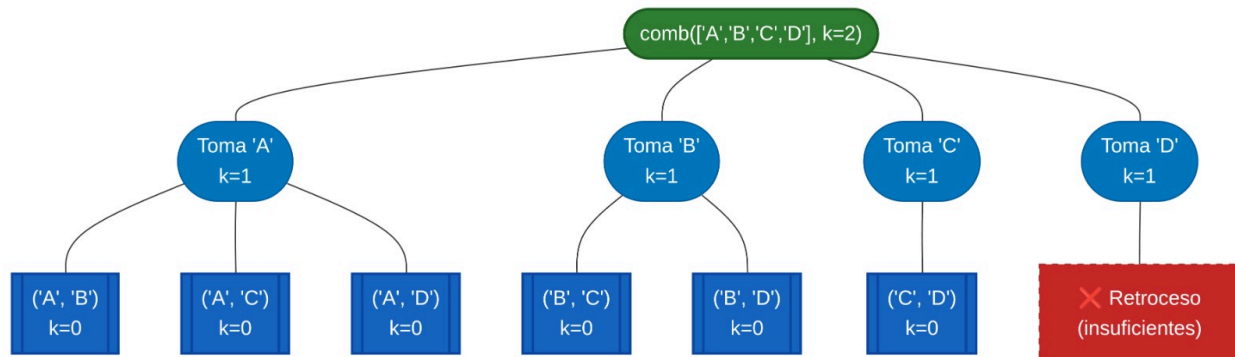
Para validar la correctitud del algoritmo, se implementó un **test contra referencia** comparando los resultados obtenidos con la función `itertools.combinations` de la biblioteca estándar de Python.

Caso de Prueba (n, k)	Coincidencia con itertools	Hojas en el Árbol
(4, 2)	Sí	6
(10, 5)	Sí	252
(20, 10)	Sí	184.756

La coincidencia elemento a elemento confirma que la implementación recursiva es funcionalmente

robusta.

Para visualizarlo mejor hicimos un diagrama de árbol, también esta imagen responde a la consigna del desafío.



4. Discusión de Límites de Cómputo y Pila (Punto d)

Uno de los puntos centrales del análisis fue la distinción entre las limitaciones del intérprete de Python y las limitaciones físicas de la máquina.

- **Profundidad de la Pila:** Para un caso como `combinaciones_rec(30, 15)`, la profundidad máxima de la pila es de solo 15 niveles (frames). Esto se encuentra muy por debajo del límite por defecto de `sys.getrecursionlimit()` (típicamente 1000), por lo que no existe riesgo de un `RecursionError` por profundidad.
- **Explosión Combinatoria:** El límite real es el **ancho del árbol**. El cálculo de `combinaciones_rec(30, 15)` arroja aproximadamente 155 millones de combinaciones. Intentar almacenar estos resultados en una lista de Python requeriría un volumen de memoria RAM estimado entre 25 y 40 GB, dependiendo del overhead de los objetos.

Se concluye que el problema para grandes valores de n y k no es la recursividad per se, sino la inviabilidad del cómputo factible debido a la complejidad exponencial del espacio de soluciones.

5. Decisiones de diseño

En un comienzo optamos por una función iterativa, pero mientras lo diseñamos en nuestra cabeza nos dimos cuenta de que era inconveniente para este caso por eso, cambiamos de idea y tomamos el camino de la recursión, de esta forma pudimos encontrar una solución que nos permitiera abarcar distintos valores de k con el mismo bloque de código.

- Para esto diseñamos una función llamada “`combinaciones_rec`”, esta posee dos parámetros, la lista que contiene los elementos a dividir (la llamamos `elementos`) y el parámetro “ k ” (indica cuántos elementos debe tener cada tupla). El primer bloque se inicia manejando el caso base, luego de esto podemos encontrar el caso de rechazo, que se ejecuta si no hay suficientes elementos, por último llegamos a la parte más interesante, que es el caso recursivo.
- El caso recursivo comienza cuando se ingresa al primer bucle `for` donde iteramos sobre los elementos de la lista, luego de buscar el elemento actual, nos posicionamos sobre él, y se vuelve a llamar a la función “`combinaciones_rec`”.
 - Pero esta vez los parámetros que pasamos son la lista desde el elemento actual más uno,

lo que abarca hasta el final de la lista, y en el segundo parámetro podemos encontrar $k-1$ (le restamos 1 para ir acercándonos cada vez más al caso base).

- Esto es así porque las funciones recursivas trabajan en forma de árbol desglosando cada llamada de sí misma hasta llegar al caso base. Dentro de este for podemos encontrar otro en el que se va armando la lista de tuplas correspondientes a cada iteración del primer. En la última línea de la función vemos el return que se ejecuta únicamente cuando los parámetros de la función en su primer llamado son " $\text{len}(\text{elementos}) > k$ " y $k \neq 0$.